Module 5

Volatility Models

Introduction

An ARCH (autoregressive conditionally heteroscedastic) model is a model for the variance of a time series. ARCH models are used to describe a changing, possibly volatile variance. Although an ARCH model could possibly be used to describe a gradually increasing variance over time, most often it is used in situations in which there may be short periods of increased variation. (Gradually increasing variance connected to a gradually increasing mean level might be better handled by transforming the variable.)

ARCH models were created in the context of econometric and finance problems having to do with the amount that investments or stocks increase (or decrease) per time period, so there's a tendency to describe them as models for that type of variable. For that reason, the variable of interest in these problems might either be $y_t = \frac{x_t - x_{t-1}}{x_{t-1}}$ the proportion gained or lost since the last time, or $log(x_t/x_{t-1}) = log(x_t) - log(x_{t-1})$ the logarithm of the ratio of this time's value to last time's value. It's not necessary that one of these be the primary variable of interest. An ARCH model could be used for any series that has periods of increased or decreased variance. This might, for example, be a property of residuals after an ARIMA model has been fit to the data.

The ARCH(1) Variance Model

Suppose that we are modeling the variance of a series y_t. The ARCH(1) model for the variance of model y_t is that conditional on y_{t-1}, the variance at time t is

$$Var(y_t|y_{t-1}) = \sigma_t^2 = \alpha_0 + \alpha_1 y_{t-1}^2. \tag{1}$$

We impose the constraints $\alpha_0 \geq 0$ and $\alpha_1 \geq 0$ to avoid negative variance.

NOTE:

The variance at time t is connected to the value of the series at time $t\breve{}1$. A relatively large value of y_{t-1}^2 gives a relatively large value of the variance at time t. This means that the value of y_t is less predictable at time $t-1$ than at times after a relatively small value ofy_{t-1}^2.

If we assume that the series has $mean = 0$ (this can always be done by centering), the ARCH model could be written as

$$y_t = \sigma_t \epsilon_t \tag{2}$$

with

$$\sigma_t = \sqrt{\alpha_0 + \alpha_1 y_{t-1}^2}$$

and

$$\epsilon_t \sim (\mu = 0, \sigma^2 = 1).$$

For inference (and maximum likelihood estimation) we would also assume that the ϵ_t are normally distributed.

Possibly Useful Results

Two potentially useful properties of the useful theoretical property of the ARCH(1) model as written in Equation (2) above are the following:

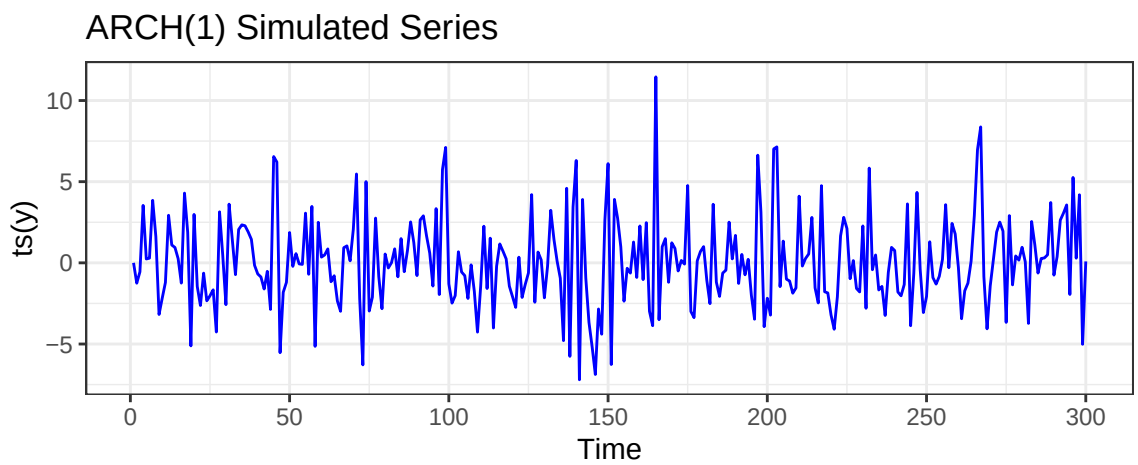
- y_t^2 has the AR(1) model, $y_t^2 = \alpha_0 + \alpha_1 y_{t-1} + error$. This model will be causal, meaning it can be converted to a legitimate infinite order MA only when $\alpha_1^2 < 1/3$.
- y_t is white noise when $0 \leq \alpha_1 \leq 1$.

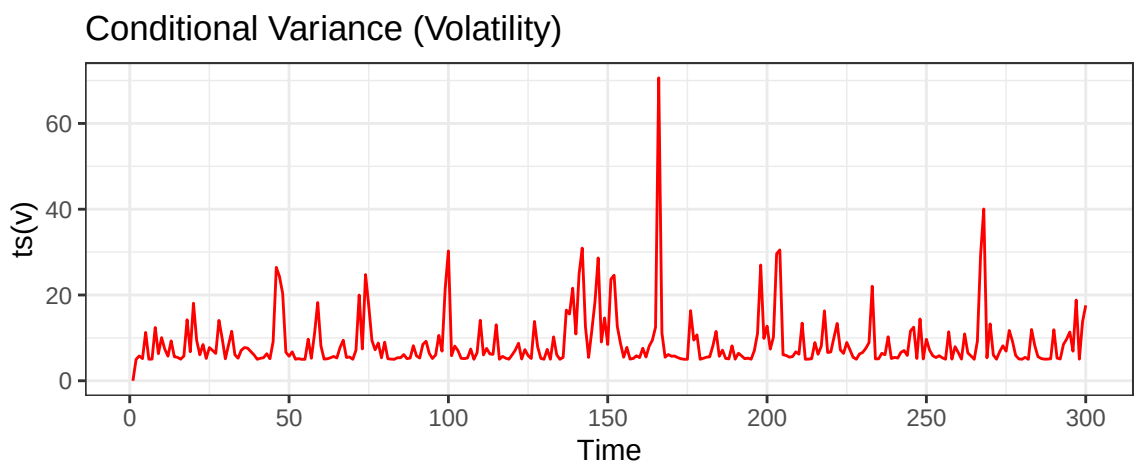
Example: ARCH Model

We simulated observations (n = 300) from the following ARCH(1) model

$$Var(y_t|y_{t-1}) = \sigma_t^2 = 5 + 0.5y_{t-1}^2.$$

The time series plot and the conditional variance plot are shown below.





Generalizations

An ARCH(m) process is one for which the variance at time t is conditional on observations at the previous m times, and the relationship is

$$Var(y_t|y_{t-1}, \cdots, y_{t-m}) = \sigma_t^2 = \alpha_0 + \alpha_1 y_{t-1}^2 + \cdots + \alpha_m y_{t-m}^2$$

With certain constraints imposed on the coefficients, the y_t series squared will theoretically be AR(m).

A GARCH (generalized autoregressive conditionally heteroscedastic) model uses values of the past squared observations and past variances to model the variance at time t. As an example, a GARCH(1,1) is

$$\sigma_t^2 = \alpha_0 + \alpha_1 y_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

In the GARCH notation, the first subscript refers to the order of the y^2 terms on the right side, and the second subscript refers to the order of the σ^2 terms.

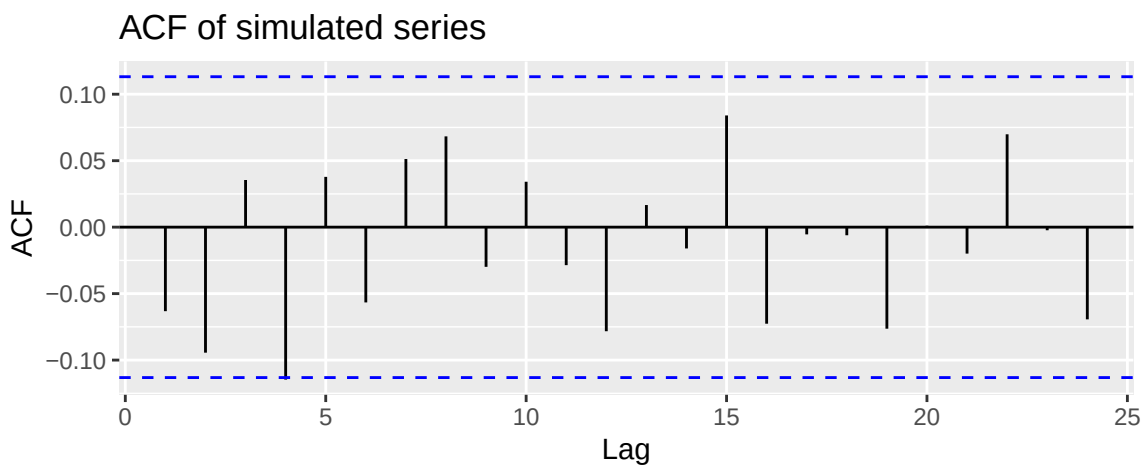
In the case of GARCH(1,1) model we we require that $\alpha_1 + \beta_1 < 1$ to ensure that the process is stationary and mean-reverting (meaning volatility reverts to a long-run average).

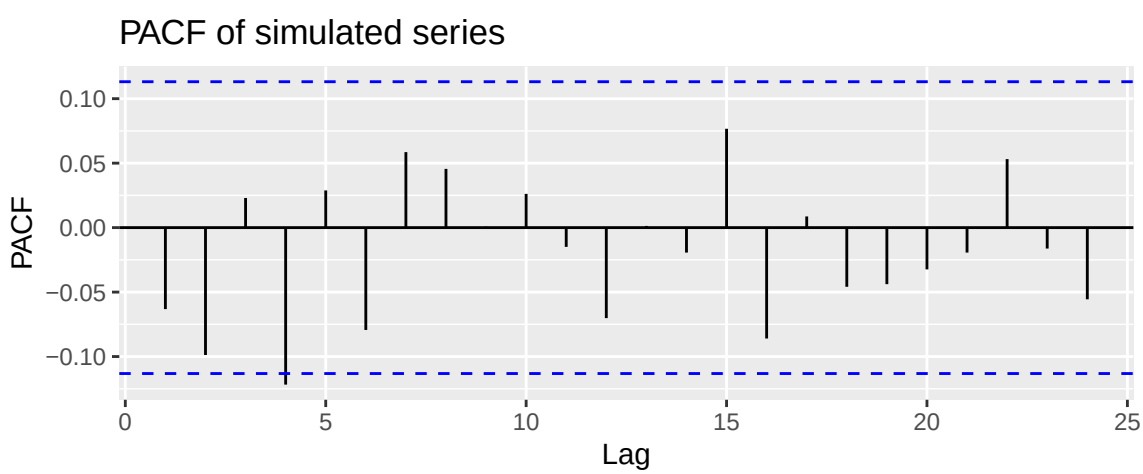
Identifying an ARCH/GARCH Model in Practice

The best identification tool may be a time series plot of the series. It's usually easy to spot periods of increased variation sprinkled through the series. It can be fruitful to look at the ACF and PACF of both y_t and y_t^2. For instance, if y_t appears to be white noise and y_t^2 appears to be AR(1), then an ARCH(1) model for the variance is suggested. If the PACF of the y_t^2 suggests AR(m), then ARCH(m) may work.

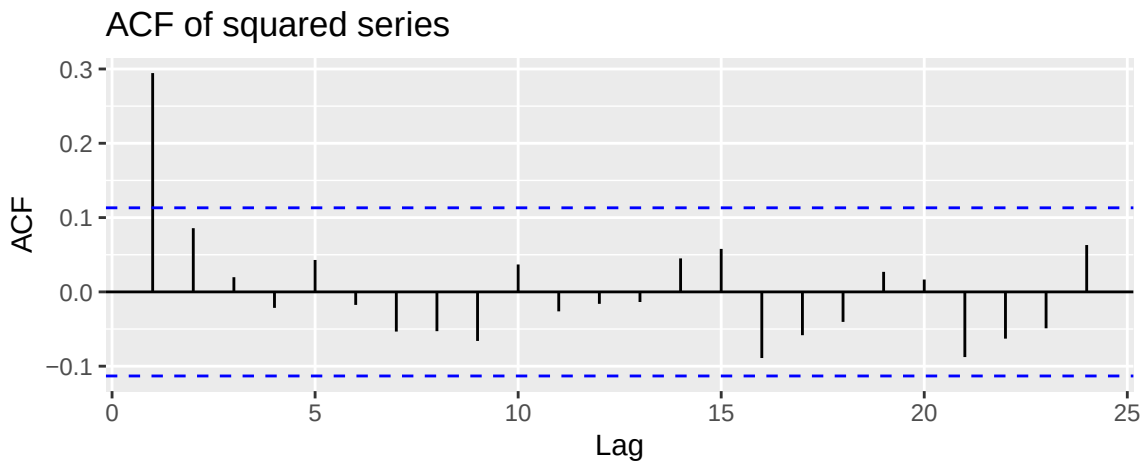
GARCH models may be suggested by an ARMA type look to the ACF and PACF of y_t^2. In practice, the ACF and PACF plots may not be suggestive of the order of ARCH of GARCH. Hence, you might have to experiment with various ARCH and GARCH structures after spotting the need in the time series plot of the series.

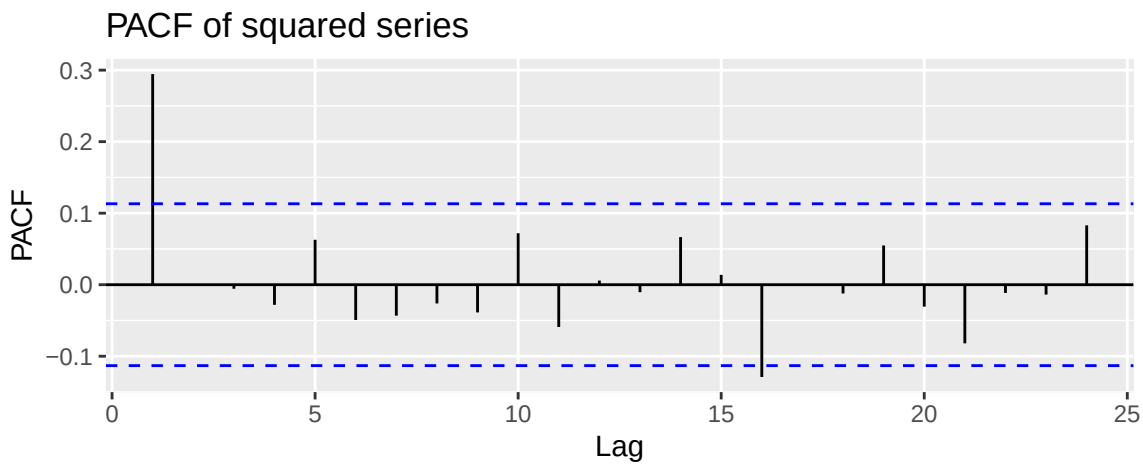
The ACF and PACF plots of the simulated data in the previous example are shown below.





While, the ACF and PACF plots of the squared series are shown below.

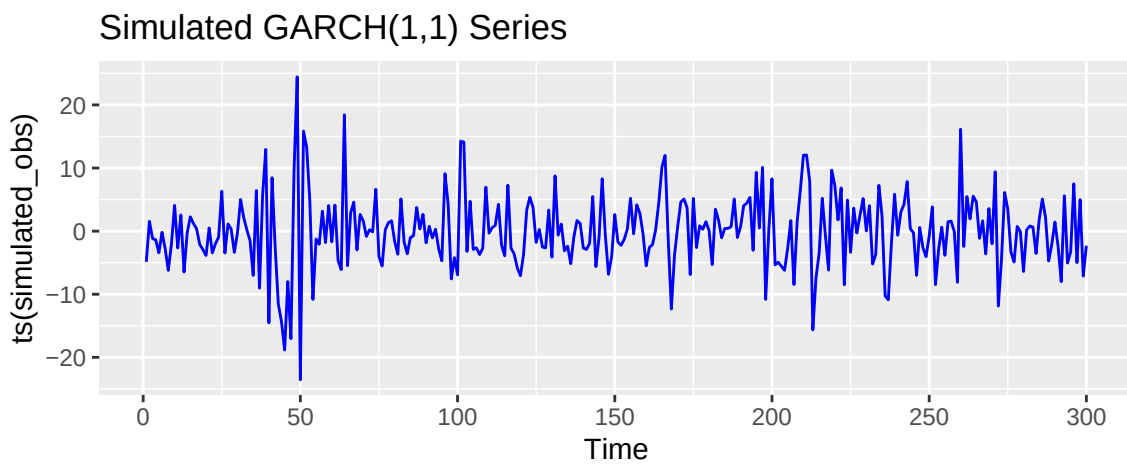


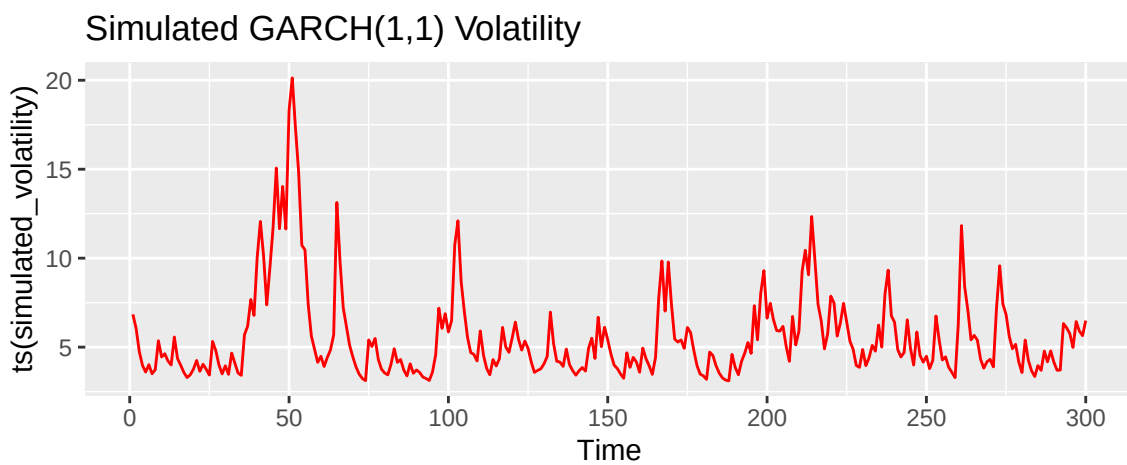


Example: GARCH Model

The following charts show the time plot and the volatility plot of simulated observation (n = 300) from the following GARCH(1,1) model.

$$Var(x_t|x_{t-1}) = \sigma_t^2 = 5 + 0.45x_{t-1} + 0.45\sigma_{t-1}^2$$





Steps in Estimating Volatility Models

The ARCH model is a popular model because its variance specifications can capture commonly observed features of the time series of financial variables. It is useful for modeling volatility and especially changes in volatility over time.

When estimating ARCH and GARCH type models, the following steps are essential:

1. Visual examination of the series
    - Examine the time series plot and check the presence of volatility.
2. Check stationarity

Before fitting an ARCH/GARCH model, the time series data (usually financial returns) must be stationary in its mean.

- Apply **ADF** test

  - *Null Hypothesis*: The time series is non-stationary (it has a unit root).
  - *Alternative Hypothesis*: The time series is stationary (no unit root).
  - *Decision Rule*: If the p-value is less than the significance level (e.g., 0.05), you reject the null hypothesis and conclude the series is stationary.

- Or, the **KPSS** test
  - *Null Hypothesis*: The time series is stationary (around a mean or linear trend).
  - *Alternative Hypothesis*: The time series is non-stationary.
  - *Decision Rule*: If the p-value is less than the significance level (e.g., 0.05), you reject the null hypothesis and conclude the series is non-stationary.

3. Check for ARCH effect

Once the mean is stationary, you need to confirm the presence of volatility clustering (ARCH effects) in the residuals before applying an ARCH/GARCH model.Apply Engle's Lagrange Multiplier (LM) test.

- *Null hypothesis*: No ARCH effect (The residuals are homoscedastic.)
- *Alternative hypothesis*: ARCH effects exist (The residuals exhibit heteroskedasticity that depends on previous lags.)

We can do this using

- *ArchTest()* fucntion in **FinTS** package
- *arch.test()* function from the **nortsTest** package

4. Estimate the ARCH/GARCH Model

Estimating ARCH/GARCH models in R is typically done using specialized packages such as **rugarch** and **fGarch**.

5. Forecasting

Variations of GARCH Model

1. Standard GARCH (): Assumes positive and negative shocks have a symmetric effect on volatility. It is the foundational model, extending ARCH by including lagged conditional variances to model volatility clustering.

2. EGARCH (Exponential GARCH): Models the logarithm of conditional variance, ensuring variance is always positive without restricting parameters. It is effective at capturing the "leverage effect," where negative shocks increase volatility more than positive shocks.

3. GJR-GARCH (Glosten-Jagannathan-Runkle GARCH): A threshold model that adds a parameter to reflect the asymmetry of news (leverage effect). It gives higher weight to negative shocks, making it superior for capturing the volatility spikes after market crashes, whereas standard GARCH fails.

4. TGARCH (Threshold GARCH): Similar to GJR-GARCH, it uses a multiplicative dummy variable in the variance equation to differentiate the effects of negative and positive shocks.

5. Other Variations: IGARCH (Integrated GARCH) for long-memory volatility, PGARCH (Power GARCH) for flexible power modeling, and GARCH-M (GARCH-in-mean) for capturing risk premiums.

An Example

In this example let us use the Google stock data that comes with the **quantmod** package.

Let us use the prices of BMW cars from June 11, 1995 to June 11, 2013.

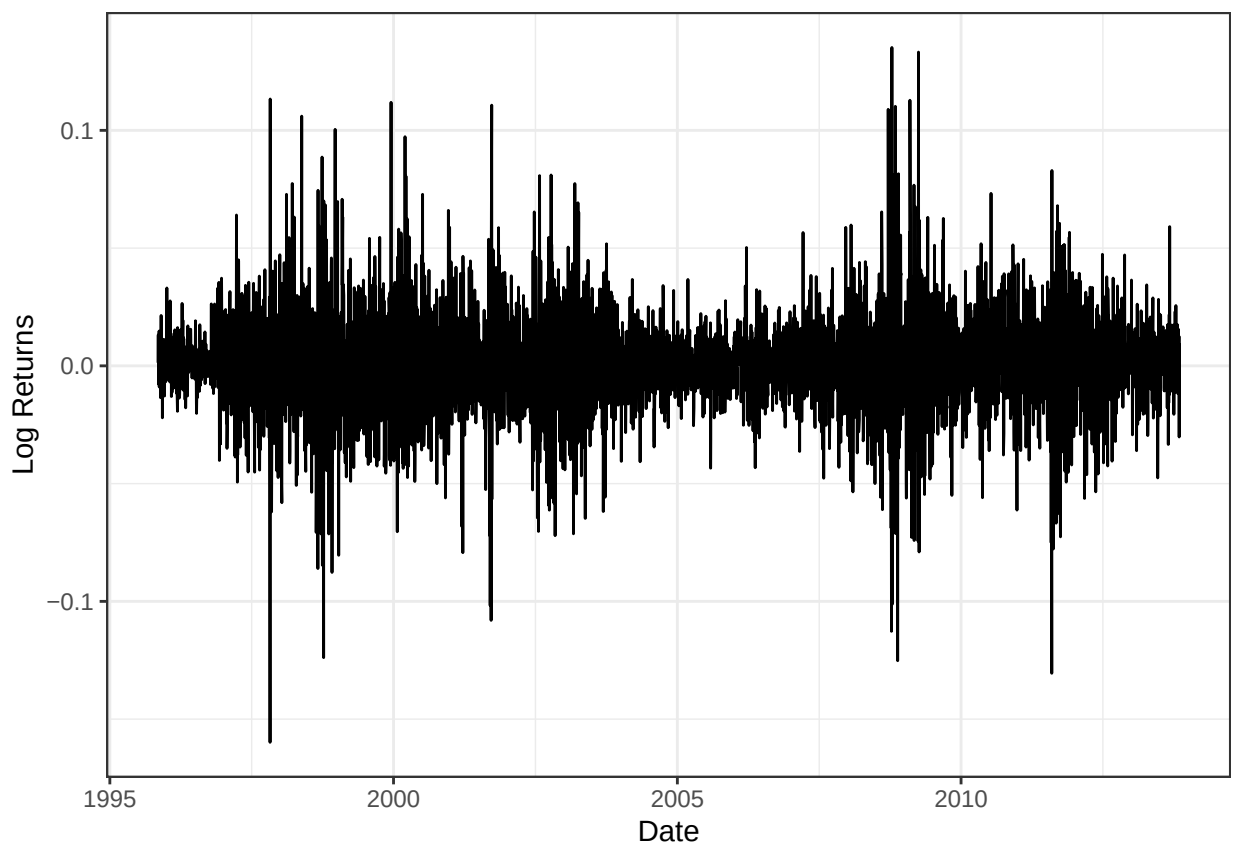
```r
bmw<-read_excel("BMW.xlsx")
head(bmw)
```

```
## # A tibble: 6 x 2
##   Years               Prices
##   <dttm>               <dbl>
## 1 1995-11-06 00:00:00   12.0
## 2 1995-11-07 00:00:00   12
## 3 1995-11-08 00:00:00   12.2
## 4 1995-11-09 00:00:00   12.3
## 5 1995-11-10 00:00:00   12.2
## 6 1995-11-13 00:00:00   12.2
```

```
#set timeseries element
bmw$years <- as.Date(bmw$Years, "%d/%m/%Y")

bmw.z = zoo(x=bmw$Prices, order.by=bmw$years)

#calculate log returns and remove first NA value
return.bmw<-Return.calculate(bmw.z, method = "log")[-1]
```

The plot of the log returns is shown below.

```
(p1 <- autoplot(return.bmw) +
  labs(x = "Date",
       y = "Log Returns") +
  theme_bw()
)
```



As shown in the plot there is clustered volatility rather than evenly distributed fluctuations. Next we check for stationarity of the log returns.

```r
adf.test(return.bmw)
```

```
## Warning in adf.test(return.bmw): p-value smaller than printed p-value
```

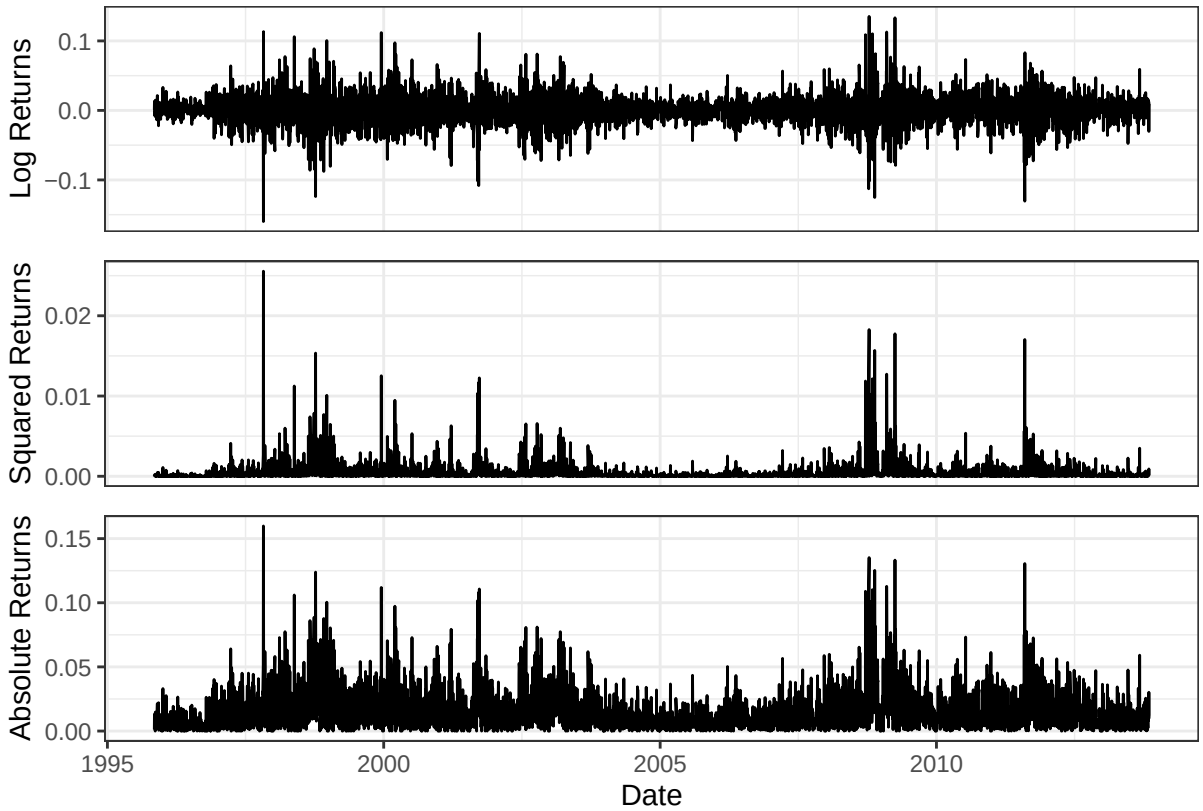
```
##
##  Augmented Dickey-Fuller Test
##
## data:  return.bmw
## Dickey-Fuller = -17.905, Lag order = 16, p-value = 0.01
## alternative hypothesis: stationary
```

The result indicates that the log returns are stationary.

We can further check the presence of volatility using time series plot of the squared log returns, and the absolute value of the log returns. These are shown below, together with the log returns plot we have seen earlier.

```r
p2 <- autoplot(return.bmw^2) +
  labs(x = "Date",
       y = "Squared Returns") +
  theme_bw()

p3 <- autoplot(abs(return.bmw)) +
  labs(x = "Date",
       y = "Absolute Returns") +
  theme_bw()
```

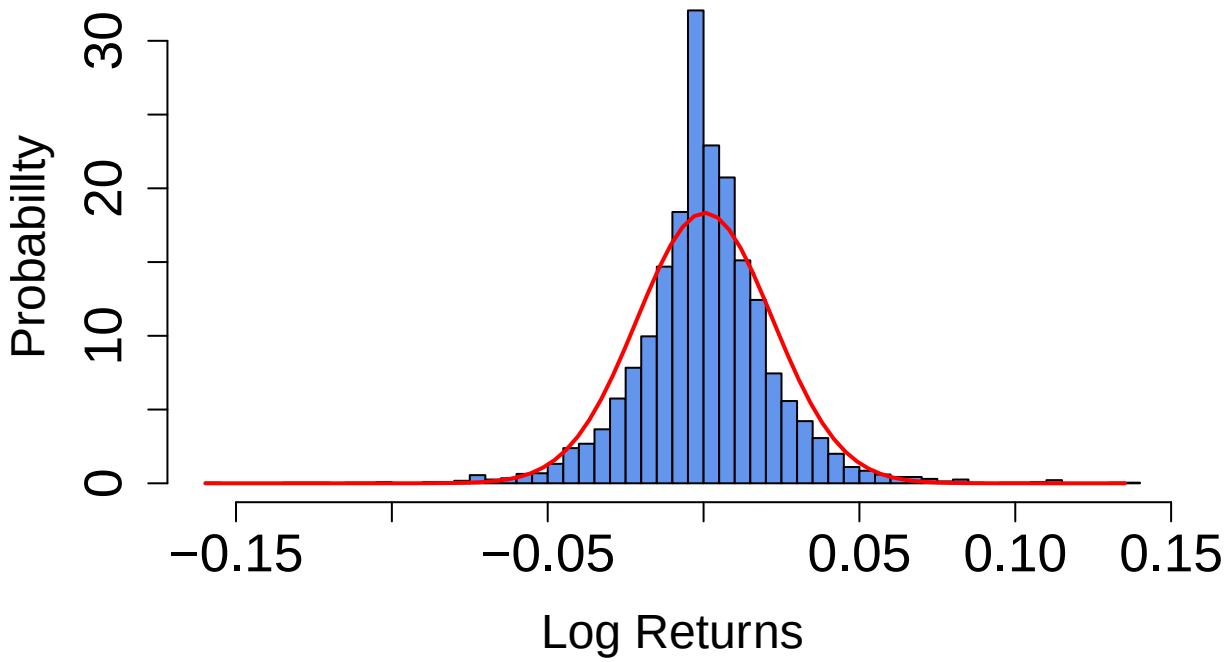
```r
p1/p2/p3 +
  plot_layout(axes = "collect")
```



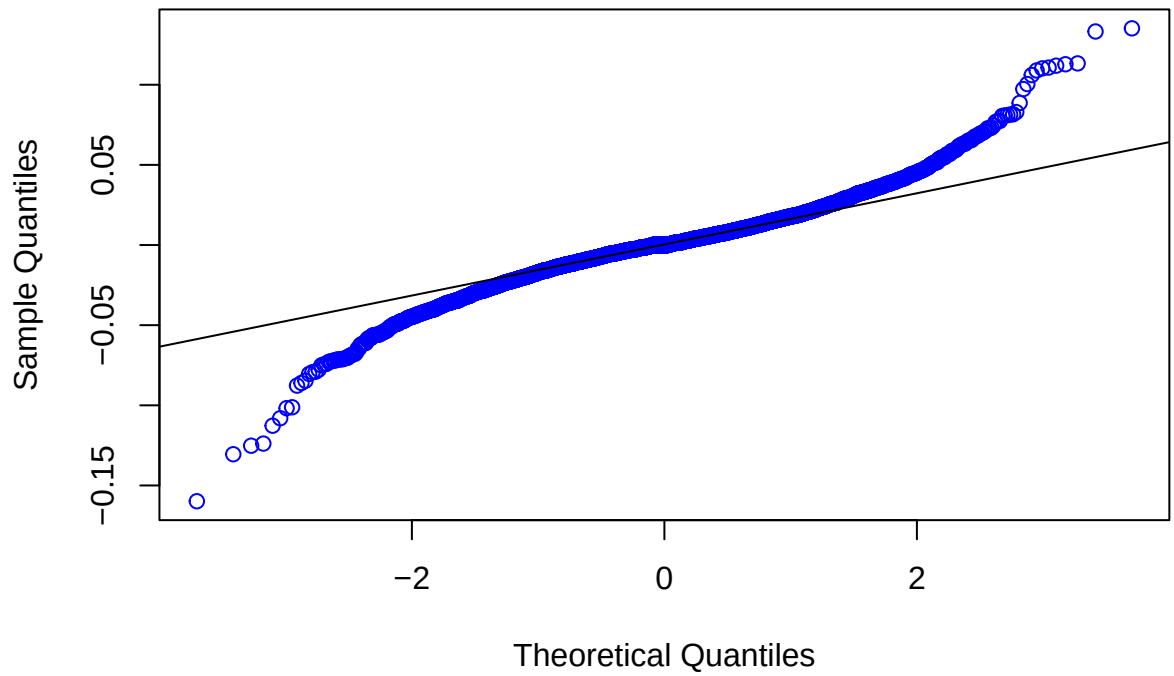
We generate the histogram and the normal probability plot below to check normality.

```
#histograme with normal density curve
hist(return.bmw,
     prob=T,
     breaks=50,
     xlab="Log Returns",
     main = "",
     ylab="Probabillty",
     col="cornflowerblue",
     cex.lab=1.5,
     cex.axis=1.7,
     cex.main=2.3)

mu<-mean(return.bmw)

sigma<-sd(return.bmw)

x<-seq(min(return.bmw),max(return.bmw),length=80)

y<-dnorm(x,mu,sigma)
```

```
lines(x,y,lwd=2,col="red")
```




```
qqnorm(return.bmw, main = "", col = "blue")
qqline(return.bmw)
```



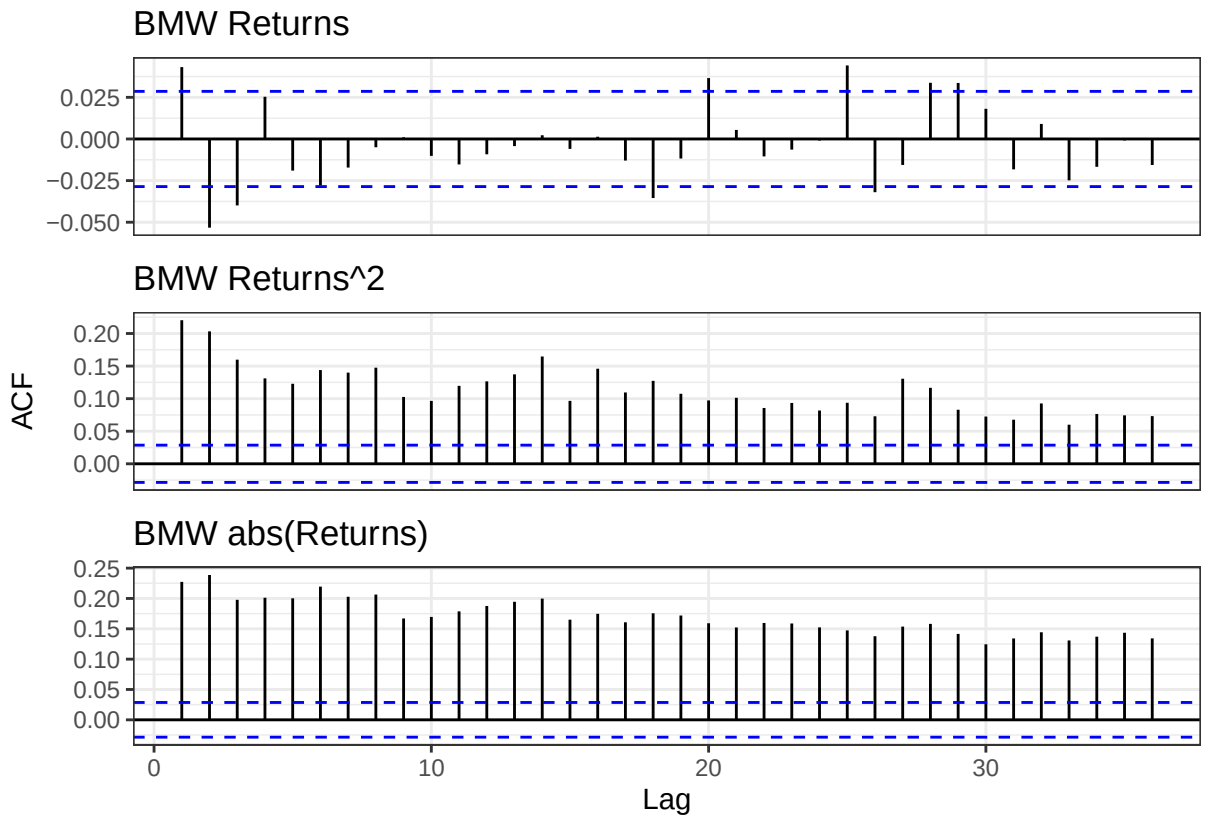

```r
jarque.bera.test(return.bmw)
```

```
##
##  Jarque Bera Test
##
## data:  return.bmw
## X-squared = 3741.4, df = 2, p-value < 2.2e-16
```

Both the histogram and the normal probability plot suggest that the log returns are not normally distributed. This is confirmed by the Jarque-Bera test.

We plot auto-correlation first to check for ARCH effect.

```r
## Convert to xts for time series features
Return.BMW<-as.xts(return.bmw)
# plot autocorrelations of returns, returns^2 and abs(returns)
p4 <- forecast::ggAcf(Return.BMW, main="BMW Returns") + theme_bw()
p5 <- forecast::ggAcf(Return.BMW^2, main="BMW Returns^2") + theme_bw()
p6 <- forecast::ggAcf(abs(Return.BMW), main="BMW abs(Returns)") + theme_bw()

p4 / p5 / p6 +
  plot_layout(axes = "collect")
```



The squared and absolute log returns show high level of auto-correlation. We can double check presence of autocorrelation in squared returns by apply Ljung-Box test on squared returns.

```
Box.test(Return.BMW^2, lag = 12, type = "Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  Return.BMW^2
## X-squared = 1222.9, df = 12, p-value < 2.2e-16
```

Indeed, autocorrelation is present.

We now proceed to testing the presence of ARCH effect using the ARCH LM test.

```
ArchTest(Return.BMW)
```

```
##
##  ARCH LM-test; Null hypothesis: no ARCH effects
##
## data:  Return.BMW
## Chi-squared = 500.01, df = 12, p-value < 2.2e-16
```

The LM test shows p-value less than 0.05 which indicates that null hypothesis (no arch effect) can be rejected. Therefore, the BMW returns exhibit ARCH effect.

We are now ready to fit an ARCH or GARCH model. For the sake of illustration, let us first fit an ARCH(1) model.

```
set.seed(1234)
#Specify the model
spec <-  ugarchspec(variance.model=list(garchOrder=c(1,0)), #ARCH(1) model
                  mean.model=list(armaOrder=c(0,0)),
                  distribution.model="norm") #Assuming normal distribution
#Fit ARCH Model
arch1.fit <- ugarchfit(data=Return.BMW, spec=spec, solver = "hybrid")
#fitted model outcome
arch1.fit
```

```
##
## *---------------------------------*
## *          GARCH Model Fit        *
## *---------------------------------*
##
## Conditional Variance Dynamics
## -----------------------------------
## GARCH Model  : sGARCH(1,0)
## Mean Model   : ARFIMA(0,0,0)
## Distribution : norm
##
## Optimal Parameters
## ------------------------------------
##         Estimate  Std. Error  t value Pr(>|t|)
## mu      0.000410    0.000291   1.4109  0.15828
## omega   0.000341    0.000010  34.7384  0.00000
## alpha1  0.306529    0.028344  10.8145  0.00000
##
## Robust Standard Errors:
##         Estimate  Std. Error  t value Pr(>|t|)
## mu      0.000410    0.000292   1.4054  0.15991
## omega   0.000341    0.000025  13.7712  0.00000
## alpha1  0.306529    0.054838   5.5897  0.00000
##
## LogLikelihood : 11489.43
##
## Information Criteria
## ------------------------------------
##
```

```
## Akaike       -4.8910
## Bayes        -4.8868
## Shibata      -4.8910
## Hannan-Quinn -4.8895
##
## Weighted Ljung-Box Test on Standardized Residuals
## ------------------------------------
##                         statistic   p-value
## Lag[1]                      10.47 0.0012133
## Lag[2*(p+q)+(p+q)-1][2]     12.12 0.0005262
## Lag[4*(p+q)+(p+q)-1][5]     16.82 0.0001436
## d.o.f=0
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## ------------------------------------
##                         statistic   p-value
## Lag[1]                      4.296 3.821e-02
## Lag[2*(p+q)+(p+q)-1][2]    55.520 2.220e-15
## Lag[4*(p+q)+(p+q)-1][5]   139.737 0.000e+00
## d.o.f=1
##
## Weighted ARCH LM Tests
## ------------------------------------
##             Statistic Shape Scale P-Value
## ARCH Lag[2]     102.4 0.500 2.000       0
## ARCH Lag[4]     157.9 1.397 1.611       0
## ARCH Lag[6]     212.0 2.222 1.500       0
##
## Nyblom stability test
## ------------------------------------
## Joint Statistic:  2.2462
## Individual Statistics:
## mu     0.3447
## omega  1.4510
## alpha1 0.6217
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:          0.846 1.01 1.35
## Individual Statistic:     0.35 0.47 0.75
##
## Sign Bias Test
## ------------------------------------
##                    t-value    prob sig
## Sign Bias            1.746 0.08089   *
```

```
## Negative Sign Bias   1.481 0.13877
## Positive Sign Bias   1.977 0.04807  **
## Joint Effect         6.103 0.10671
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## ------------------------------------
##   group statistic p-value(g-1)
## 1    20     410.0    3.582e-75
## 2    30     459.1    7.047e-79
## 3    40     501.5    1.217e-81
## 4    50     597.3    2.552e-95
##
##
## Elapsed time : 0.870549
```

There is still significant autocorrelation in both the standardized residuals and the squared standardized residuals, suggesting poor fit of the ARCH(1) model.

Let us try fitting a GARCH(1,1) model.

```r
set.seed(1234)
#Specify the model
spec <-  ugarchspec(variance.model=list(garchOrder=c(1,1)),
                  mean.model=list(armaOrder=c(0,0)),
                  distribution.model="norm") #Assuming normal distribution
#Fit ARCH Model
garch11.fit <- ugarchfit(data=Return.BMW, spec=spec)
#fitted model outcome
garch11.fit
```

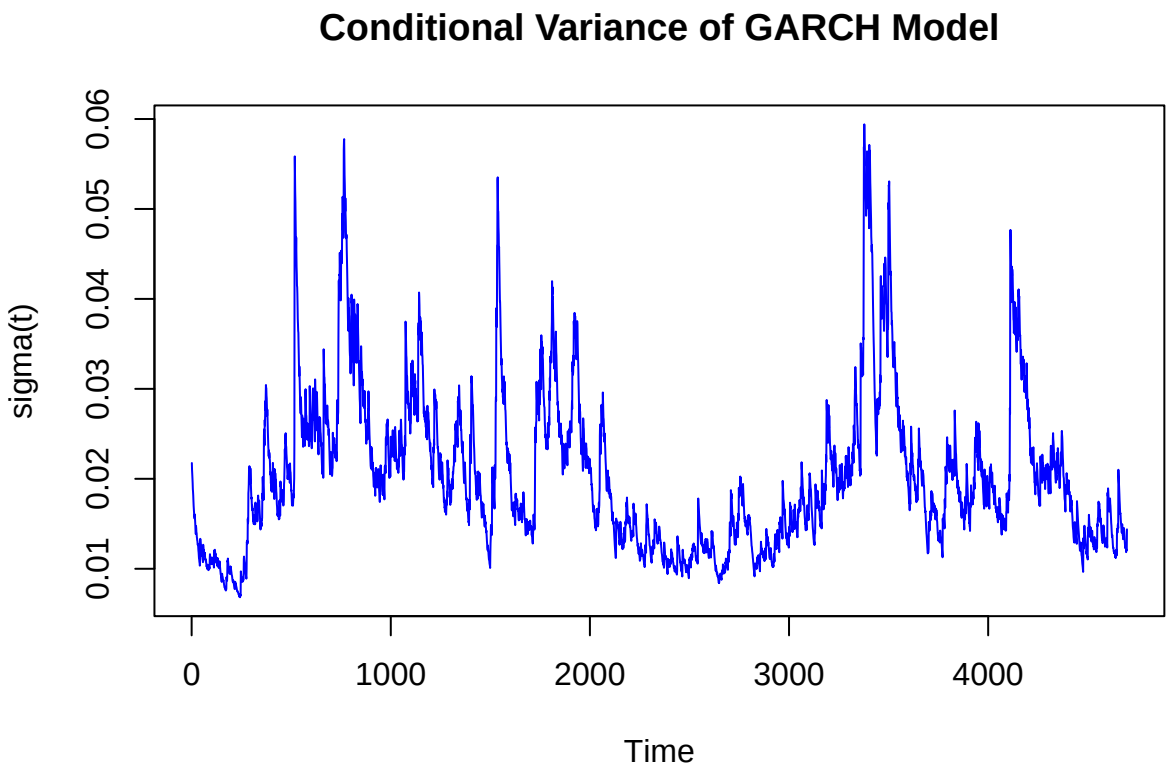
```
##
## *---------------------------------*
## *          GARCH Model Fit        *
## *---------------------------------*
##
## Conditional Variance Dynamics
## -----------------------------------
## GARCH Model  : sGARCH(1,1)
## Mean Model   : ARFIMA(0,0,0)
## Distribution : norm
##
## Optimal Parameters
## ------------------------------------
##         Estimate  Std. Error  t value Pr(>|t|)
```

```
## mu      0.000782    0.000232   3.3717 0.000747
## omega   0.000002    0.000002   1.0704 0.284421
## alpha1  0.071735    0.015020   4.7759 0.000002
## beta1   0.926360    0.015309  60.5089 0.000000
##
## Robust Standard Errors:
##         Estimate  Std. Error  t value Pr(>|t|)
## mu      0.000782    0.000318  2.45492 0.014092
## omega   0.000002    0.000016  0.14397 0.885521
## alpha1  0.071735    0.105517  0.67984 0.496606
## beta1   0.926360    0.109274  8.47742 0.000000
##
## LogLikelihood : 12002.45
##
## Information Criteria
## ------------------------------------
##
## Akaike       -5.1090
## Bayes        -5.1035
## Shibata      -5.1090
## Hannan-Quinn -5.1071
##
## Weighted Ljung-Box Test on Standardized Residuals
## ------------------------------------
##                         statistic   p-value
## Lag[1]                      11.44 0.0007188
## Lag[2*(p+q)+(p+q)-1][2]     12.15 0.0005181
## Lag[4*(p+q)+(p+q)-1][5]     15.09 0.0004207
## d.o.f=0
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                      3.214 0.07301
## Lag[2*(p+q)+(p+q)-1][5]     3.573 0.31228
## Lag[4*(p+q)+(p+q)-1][9]     5.816 0.31986
## d.o.f=2
##
## Weighted ARCH LM Tests
## ------------------------------------
##             Statistic Shape Scale P-Value
## ARCH Lag[3]    0.1413 0.500 2.000  0.7069
## ARCH Lag[5]    0.1936 1.440 1.667  0.9670
## ARCH Lag[7]    2.5965 2.315 1.543  0.5927
```

```
##
## Nyblom stability test
## ------------------------------------
## Joint Statistic:  22.0022
## Individual Statistics:
## mu     0.11762
## omega  7.07254
## alpha1 0.07861
## beta1  0.07332
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:          1.07 1.24 1.6
## Individual Statistic:     0.35 0.47 0.75
##
## Sign Bias Test
## ------------------------------------
##                    t-value      prob sig
## Sign Bias            3.137 1.716e-03 ***
## Negative Sign Bias   3.628 2.885e-04 ***
## Positive Sign Bias   2.901 3.740e-03 ***
## Joint Effect        21.591 7.934e-05 ***
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## ------------------------------------
##   group statistic p-value(g-1)
## 1    20     192.5    1.042e-30
## 2    30     229.8    3.948e-33
## 3    40     244.5    1.424e-31
## 4    50     257.5    4.520e-30
##
##
## Elapsed time : 0.4124448
```

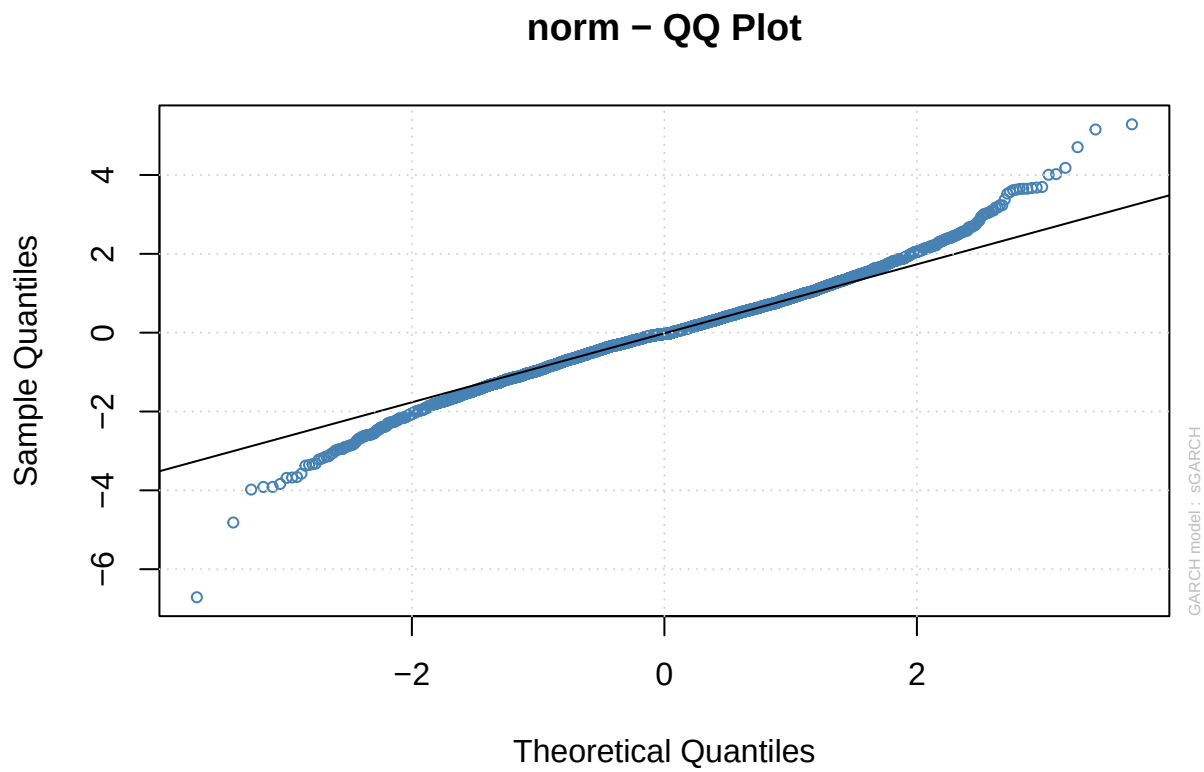
The Ljung-Box test indicates no evidence of serial correlation in the squared residuals of the GARCH model.

```r
#Conditional Volatility Plot for GARCH Model
plot.ts(sigma(garch11.fit),
        ylab="sigma(t)",
        col="blue",
        main = "Conditional Variance of GARCH Model")
```



The plot demonstrates significant volatility clustering and mean-reverting behavior. The sharp spikes indicate the model's sensitivity to market shocks, while the subsequent decays illustrate the persistence of volatility over time.

```
##QQ-plot for fitted GARCH Model
plot(garch11.fit, which=9)#displays the QQ plot
```



These results suggest that ARCH(1) and GARCH(1,1) with assuming normal distribution are not appropriate models.

Hence, we fit ARCH(1) and GARCH(1,1) assuming a t-distribution.

```
set.seed(1234)
#Specify the model
spec = ugarchspec(variance.model=list(garchOrder=c(1,0)),
                  mean.model=list(armaOrder=c(0,0)),
                  distribution.model="std") # Specifies a t distribution
#Fit ARCH Model
arch1.fit.t=ugarchfit(data=Return.BMW,spec=spec, solver = "hybrid")
arch1.fit.t
```

```
##
## *---------------------------------*
## *          GARCH Model Fit        *
## *---------------------------------*
##
## Conditional Variance Dynamics
## -----------------------------------
## GARCH Model  : sGARCH(1,0)
```

```
## Mean Model   : ARFIMA(0,0,0)
## Distribution : std
##
## Optimal Parameters
## ------------------------------------
##         Estimate  Std. Error  t value Pr(>|t|)
## mu      0.000607    0.000246   2.4625 0.013796
## omega   0.000362    0.000021  17.1841 0.000000
## alpha1  0.376501    0.046961   8.0173 0.000000
## shape   3.718663    0.233005  15.9596 0.000000
##
## Robust Standard Errors:
##         Estimate  Std. Error  t value Pr(>|t|)
## mu      0.000607    0.000220   2.7536 0.005894
## omega   0.000362    0.000028  12.9959 0.000000
## alpha1  0.376501    0.067357   5.5896 0.000000
## shape   3.718663    0.268848  13.8318 0.000000
##
## LogLikelihood : 11765.11
##
## Information Criteria
## ------------------------------------
##
## Akaike       -5.0079
## Bayes        -5.0024
## Shibata      -5.0079
## Hannan-Quinn -5.0060
##
## Weighted Ljung-Box Test on Standardized Residuals
## ------------------------------------
##                         statistic   p-value
## Lag[1]                      10.04 0.0015330
## Lag[2*(p+q)+(p+q)-1][2]     11.53 0.0007518
## Lag[4*(p+q)+(p+q)-1][5]     16.07 0.0002293
## d.o.f=0
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## ------------------------------------
##                         statistic   p-value
## Lag[1]                      6.984 8.224e-03
## Lag[2*(p+q)+(p+q)-1][2]    55.905 1.776e-15
## Lag[4*(p+q)+(p+q)-1][5]   134.093 0.000e+00
## d.o.f=1
##
```

```
## Weighted ARCH LM Tests
## ------------------------------------
##             Statistic Shape Scale P-Value
## ARCH Lag[2]     97.76 0.500 2.000       0
## ARCH Lag[4]    148.05 1.397 1.611       0
## ARCH Lag[6]    198.60 2.222 1.500       0
##
## Nyblom stability test
## ------------------------------------
## Joint Statistic:  3.5924
## Individual Statistics:
## mu     0.1155
## omega  2.7352
## alpha1 1.0005
## shape  2.1233
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:          1.07 1.24 1.6
## Individual Statistic:     0.35 0.47 0.75
##
## Sign Bias Test
## ------------------------------------
##                    t-value     prob sig
## Sign Bias           2.1241 0.033713  **
## Negative Sign Bias  0.9751 0.329543
## Positive Sign Bias  2.6201 0.008819 ***
## Joint Effect        7.9961 0.046091  **
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## ------------------------------------
##   group statistic p-value(g-1)
## 1    20     125.9    8.780e-18
## 2    30     181.9    4.469e-24
## 3    40     291.1    2.636e-40
## 4    50     426.2    1.319e-61
##
##
## Elapsed time : 0.777451
```

```r
set.seed(1234)
#Specify the model
spec <-  ugarchspec(variance.model=list(garchOrder=c(1,1)),
                    mean.model=list(armaOrder=c(0,0)),
                    distribution.model="std")
```

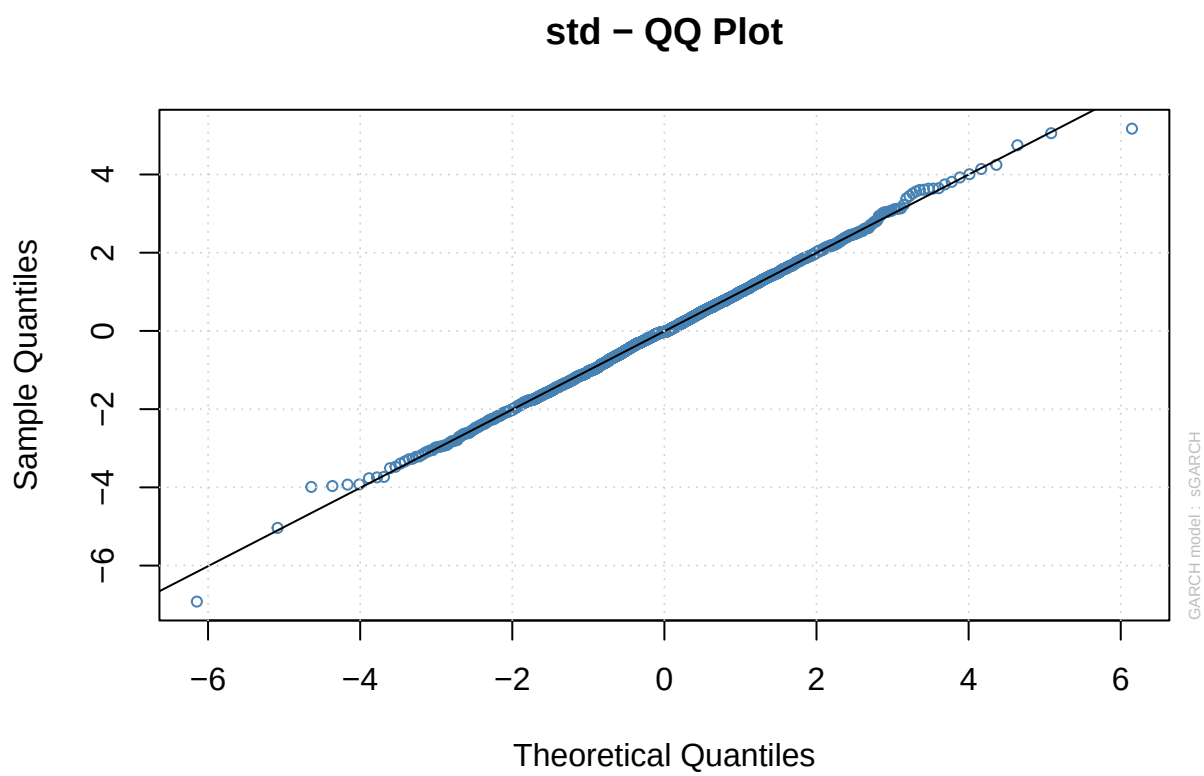
```r
#Fit ARCH Model
garch11.fit.t <- ugarchfit(data=Return.BMW, spec=spec)
#fitted model outcome
garch11.fit.t
```

```
##
## *---------------------------------*
## *          GARCH Model Fit        *
## *---------------------------------*
##
## Conditional Variance Dynamics
## -----------------------------------
## GARCH Model  : sGARCH(1,1)
## Mean Model   : ARFIMA(0,0,0)
## Distribution : std
##
## Optimal Parameters
## ------------------------------------
##         Estimate  Std. Error  t value Pr(>|t|)
## mu      0.000622    0.000214   2.8998 0.003734
## omega   0.000001    0.000001   1.3129 0.189229
## alpha1  0.061530    0.008368   7.3528 0.000000
## beta1   0.937470    0.008733 107.3434 0.000000
## shape   6.529129    0.580435  11.2487 0.000000
##
## Robust Standard Errors:
##         Estimate  Std. Error  t value Pr(>|t|)
## mu      0.000622    0.000202  3.07369 0.002114
## omega   0.000001    0.000004  0.37824 0.705256
## alpha1  0.061530    0.036174  1.70093 0.088956
## beta1   0.937470    0.036168 25.91982 0.000000
## shape   6.529129    0.741440  8.80601 0.000000
##
## LogLikelihood : 12107.4
##
## Information Criteria
## ------------------------------------
##
## Akaike       -5.1532
## Bayes        -5.1464
## Shibata      -5.1533
## Hannan-Quinn -5.1508
##
## Weighted Ljung-Box Test on Standardized Residuals
```

```
## ------------------------------------
##                          statistic   p-value
## Lag[1]                       11.05 0.0008854
## Lag[2*(p+q)+(p+q)-1][2]      11.75 0.0006583
## Lag[4*(p+q)+(p+q)-1][5]      14.62 0.0005606
## d.o.f=0
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## ------------------------------------
##                          statistic p-value
## Lag[1]                       6.344 0.01178
## Lag[2*(p+q)+(p+q)-1][5]      7.123 0.04842
## Lag[4*(p+q)+(p+q)-1][9]     10.013 0.04985
## d.o.f=2
##
## Weighted ARCH LM Tests
## ------------------------------------
##             Statistic Shape Scale P-Value
## ARCH Lag[3] 0.0003185 0.500 2.000  0.9858
## ARCH Lag[5] 0.0214647 1.440 1.667  0.9985
## ARCH Lag[7] 3.1536273 2.315 1.543  0.4850
##
## Nyblom stability test
## ------------------------------------
## Joint Statistic:  74.552
## Individual Statistics:
## mu      0.09023
## omega  19.28282
## alpha1  0.22503
## beta1   0.11457
## shape   0.12058
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:          1.28 1.47 1.88
## Individual Statistic:     0.35 0.47 0.75
##
## Sign Bias Test
## ------------------------------------
##                    t-value      prob sig
## Sign Bias            3.436 5.966e-04 ***
## Negative Sign Bias   4.135 3.605e-05 ***
## Positive Sign Bias   2.859 4.263e-03 ***
## Joint Effect        25.278 1.350e-05 ***
##
```

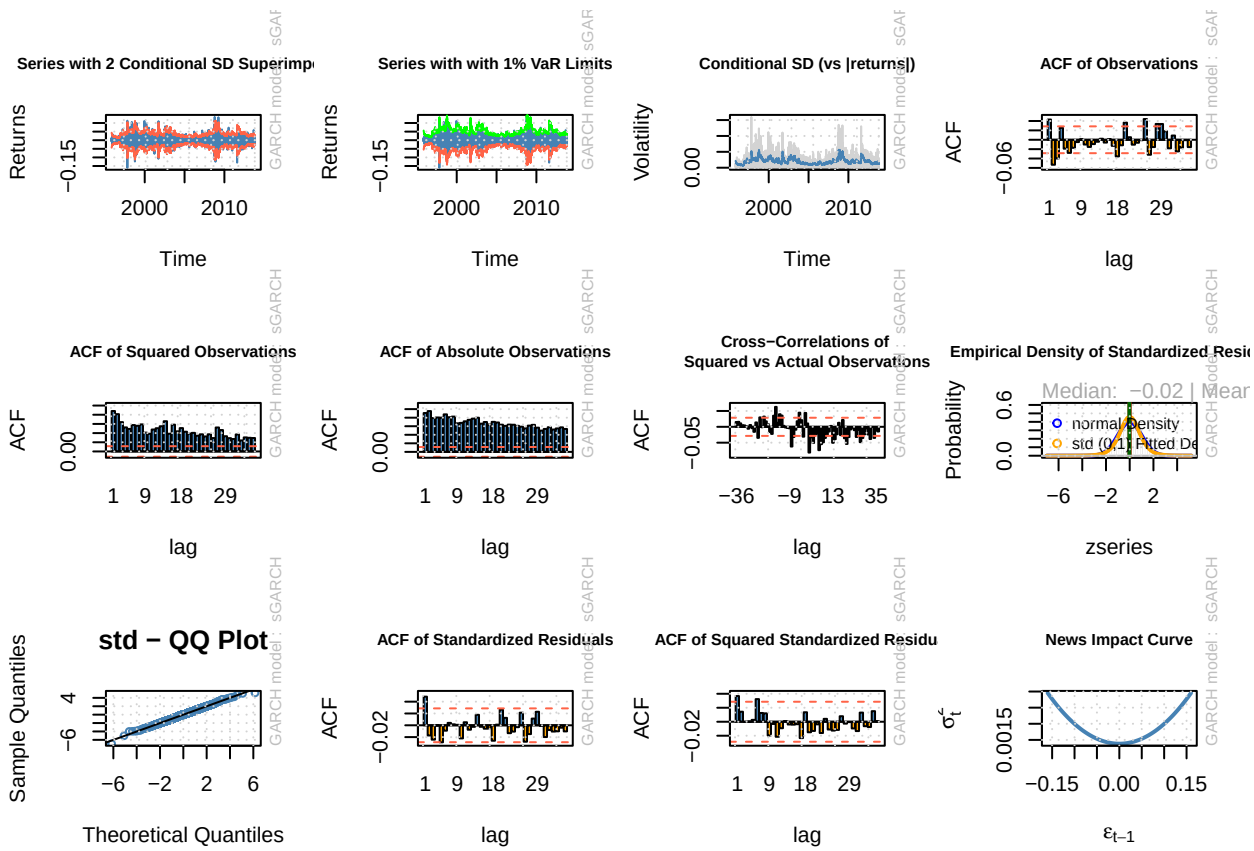
```
##
## Adjusted Pearson Goodness-of-Fit Test:
## ------------------------------------
##   group statistic p-value(g-1)
## 1    20     120.7    8.173e-17
## 2    30     147.3    8.856e-18
## 3    40     182.3    2.128e-20
## 4    50     175.8    3.533e-16
##
##
## Elapsed time : 0.747246
```

```
plot(garch11.fit.t, which=9)
```




```
plot(garch11.fit.t, which= "all")
```

```
##
## please wait...calculating quantiles...
```




```
#Model selection usinh information critarion
model.list = list("ARCH(1)" = arch1.fit,
                  "ARCH(1)-t" = arch1.fit.t,
                  "GARCH(1,1)" = garch11.fit,
                  "GARCH(1,1)-t" = garch11.fit.t)

info.mat = sapply(model.list, infocriteria)

rownames(info.mat) = rownames(infocriteria(garch11.fit))

info.mat
```

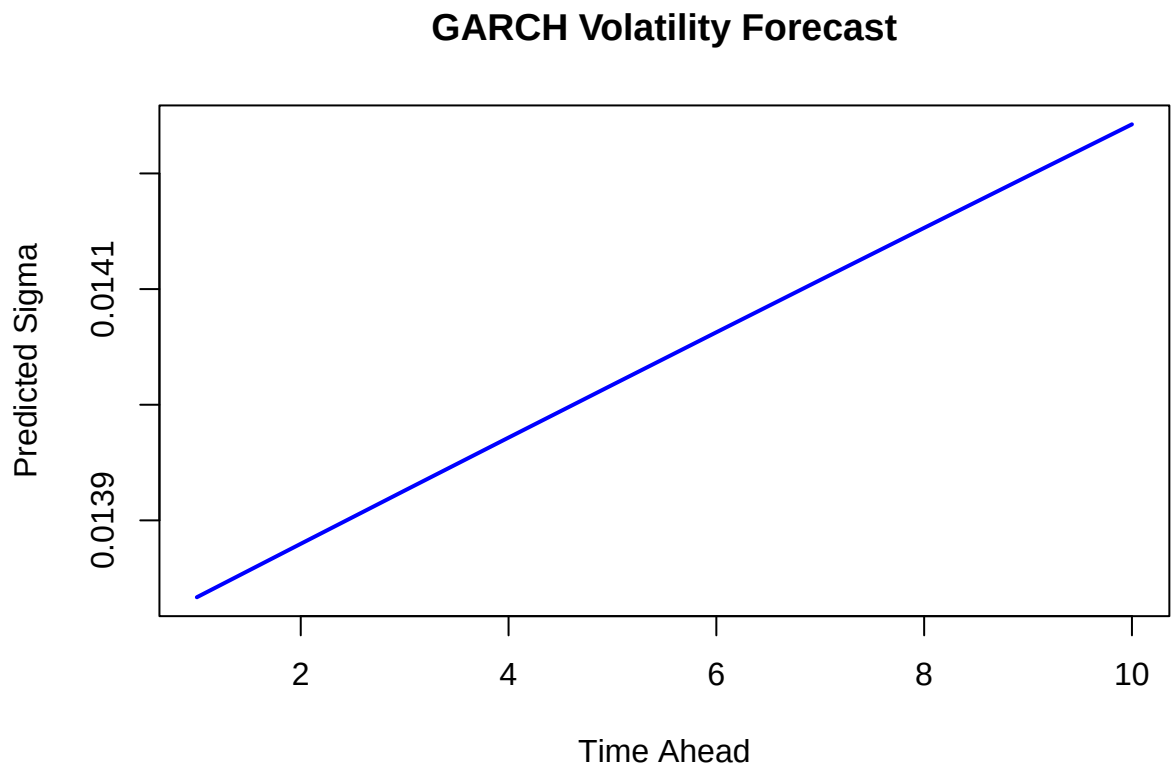
```
##                ARCH(1) ARCH(1)-t GARCH(1,1) GARCH(1,1)-t
## Akaike       -4.890963 -5.007926  -5.108986    -5.153248
## Bayes        -4.886841 -5.002429  -5.103490    -5.146377
## Shibata      -4.890964 -5.007928  -5.108988    -5.153251
## Hannan-Quinn -4.889514 -5.005994  -5.107054    -5.150833
```

GARCH(1,1) assuming a t distribution provides the best fit among the four models tested. The code chunk below generates a 10-period forecast.

```r
# 1. Generate the forecast for the next 10 periods (e.g., 10 days)
garch.forecast <- ugarchforecast(garch11.fit.t, n.ahead = 10)

# 2. Look at the numerical results
print(garch.forecast)
```

```
##
## *------------------------------------*
## *       GARCH Model Forecast         *
## *------------------------------------*
## Model: sGARCH
## Horizon: 10
## Roll Steps: 0
## Out of Sample: 0
##
## 0-roll forecast [T0=2013-11-06]:
##         Series   Sigma
## T+1  0.0006219 0.01383
## T+2  0.0006219 0.01388
## T+3  0.0006219 0.01393
## T+4  0.0006219 0.01397
## T+5  0.0006219 0.01402
## T+6  0.0006219 0.01406
## T+7  0.0006219 0.01411
## T+8  0.0006219 0.01415
## T+9  0.0006219 0.01420
## T+10 0.0006219 0.01424
```

```r
# 3. Extract the sigma (volatility) forecast values
fore_sigma <- as.numeric(sigma(garch.forecast))

# 4. Create a simple plot
plot(fore_sigma, type = "l", col = "blue", lwd = 2,
     main = "GARCH Volatility Forecast",
     xlab = "Time Ahead", ylab = "Predicted Sigma")
```



We can also combine ARIMA with GARCH. For example, we can use the *auto.arima()* to find a ARIMA model for the log returns. This is shown below.

```
forecast::auto.arima(Return.BMW)
```

```
## Series: Return.BMW
## ARIMA(1,0,5) with non-zero mean
##
## Coefficients:
##          ar1      ma1      ma2      ma3     ma4      ma5   mean
##       0.8241  -0.7808  -0.0870  -0.0012  0.0586  -0.0423  4e-04
## s.e.  0.1024   0.1031   0.0191   0.0193  0.0191   0.0153  3e-04
##
## sigma^2 = 0.0004698:  log likelihood = 11335.93
## AIC=-22655.86   AICc=-22655.83   BIC=-22604.23
```

We then combine the ARMA(1,5) with GARCH(1,1) as follows.

```
set.seed(1234)
#Specify the model
spec <-  ugarchspec(variance.model=list(garchOrder=c(1,1)),
```

```r
                  mean.model=list(armaOrder=c(1,5)),
                  distribution.model="std")
#Fit ARCH Model
arma15.garch11.fit.t <- ugarchfit(data=Return.BMW, spec=spec)
#fitted model outcome
arma15.garch11.fit.t
```

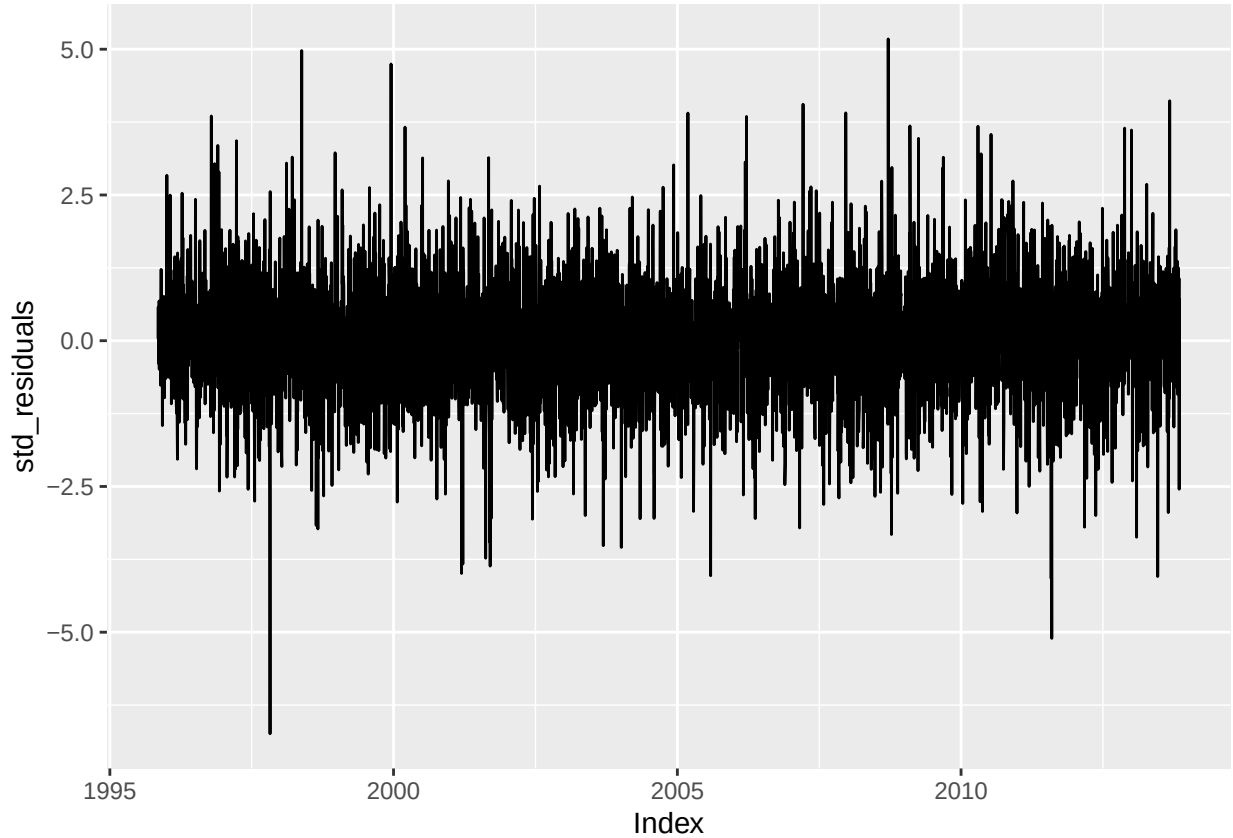
```
##
## *---------------------------------*
## *          GARCH Model Fit        *
## *---------------------------------*
##
## Conditional Variance Dynamics
## -----------------------------------
## GARCH Model  : sGARCH(1,1)
## Mean Model   : ARFIMA(1,0,5)
## Distribution : std
##
## Optimal Parameters
## ------------------------------------
##         Estimate  Std. Error  t value Pr(>|t|)
## mu      0.000638    0.000172   3.6996 0.000216
## ar1     0.879482    0.089890   9.7840 0.000000
## ma1    -0.844931    0.089871  -9.4016 0.000000
## ma2    -0.058522    0.019751  -2.9630 0.003046
## ma3    -0.010524    0.018979  -0.5545 0.579235
## ma4     0.031774    0.019444   1.6341 0.102235
## ma5    -0.021460    0.016430  -1.3061 0.191517
## omega   0.000001    0.000001   1.3598 0.173889
## alpha1  0.061862    0.008350   7.4088 0.000000
## beta1   0.937138    0.008706 107.6442 0.000000
## shape   6.541266    0.585958  11.1634 0.000000
##
## Robust Standard Errors:
##         Estimate  Std. Error  t value Pr(>|t|)
## mu      0.000638    0.000181  3.52869 0.000418
## ar1     0.879482    0.114575  7.67602 0.000000
## ma1    -0.844931    0.113310 -7.45682 0.000000
## ma2    -0.058522    0.020437 -2.86346 0.004190
## ma3    -0.010524    0.019377 -0.54310 0.587058
## ma4     0.031774    0.019627  1.61892 0.105466
## ma5    -0.021460    0.018313 -1.17184 0.241262
## omega   0.000001    0.000004  0.39804 0.690597
## alpha1  0.061862    0.035546  1.74034 0.081800
```

```
## beta1   0.937138    0.035478 26.41440 0.000000
## shape   6.541266    0.731274  8.94503 0.000000
##
## LogLikelihood : 12118.24
##
## Information Criteria
## ------------------------------------
##
## Akaike       -5.1553
## Bayes        -5.1402
## Shibata      -5.1553
## Hannan-Quinn -5.1500
##
## Weighted Ljung-Box Test on Standardized Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                      1.424  0.2327
## Lag[2*(p+q)+(p+q)-1][17]    5.982  1.0000
## Lag[4*(p+q)+(p+q)-1][29]   11.853  0.8667
## d.o.f=6
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                      6.481 0.01090
## Lag[2*(p+q)+(p+q)-1][5]     7.938 0.03053
## Lag[4*(p+q)+(p+q)-1][9]    10.967 0.03092
## d.o.f=2
##
## Weighted ARCH LM Tests
## ------------------------------------
##             Statistic Shape Scale P-Value
## ARCH Lag[3]   0.01008 0.500 2.000  0.9200
## ARCH Lag[5]   0.02237 1.440 1.667  0.9984
## ARCH Lag[7]   3.23033 2.315 1.543  0.4712
##
## Nyblom stability test
## ------------------------------------
## Joint Statistic:  73.6635
## Individual Statistics:
## mu      0.14332
## ar1     0.05547
## ma1     0.07501
## ma2     0.05206
```

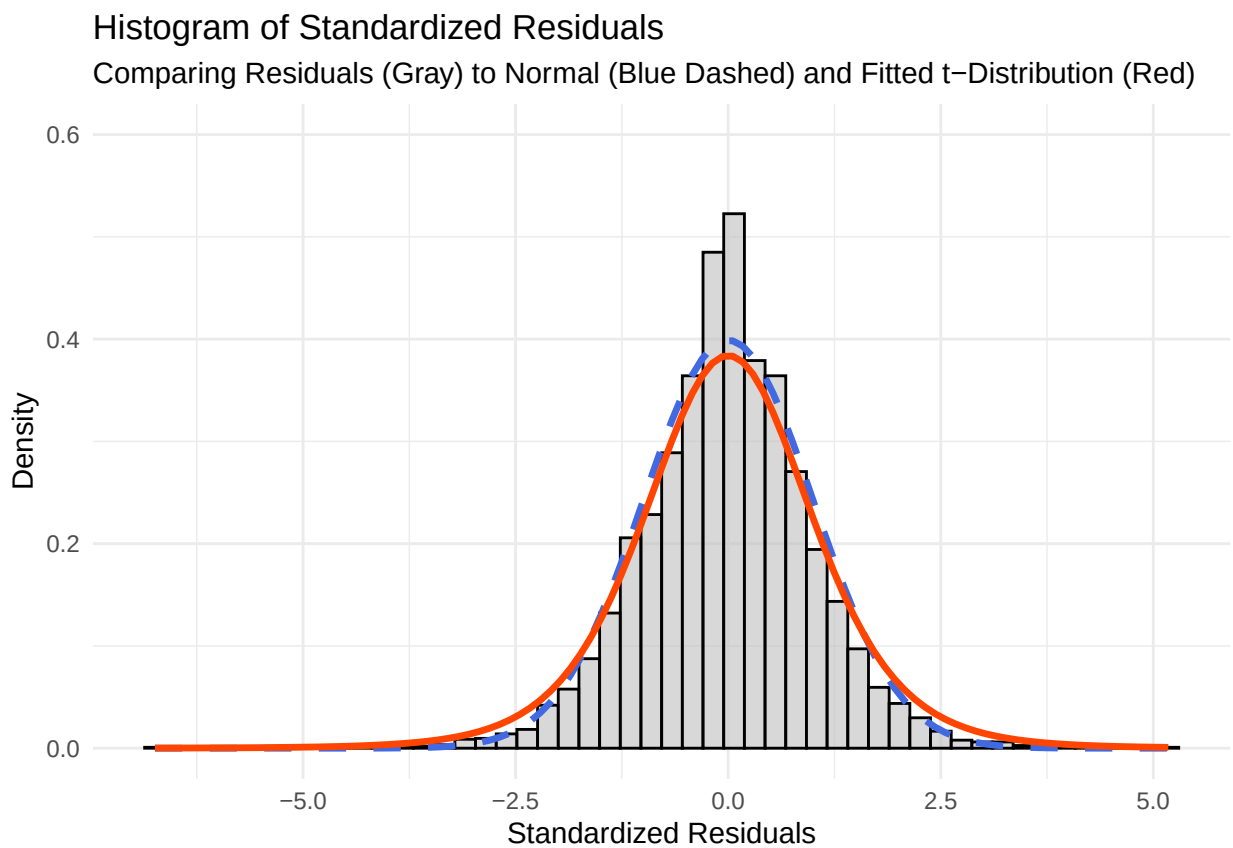
```
## ma3     0.06323
## ma4     0.08617
## ma5     0.08093
## omega  18.99119
## alpha1  0.22102
## beta1   0.10727
## shape   0.11290
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:          2.49 2.75 3.27
## Individual Statistic:     0.35 0.47 0.75
##
## Sign Bias Test
## ------------------------------------
##                    t-value      prob sig
## Sign Bias            2.484 0.0130342  **
## Negative Sign Bias   3.709 0.0002105 ***
## Positive Sign Bias   2.379 0.0173845  **
## Joint Effect        19.963 0.0001727 ***
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## ------------------------------------
##   group statistic p-value(g-1)
## 1    20     63.29    1.157e-06
## 2    30     68.36    5.061e-05
## 3    40     92.91    2.715e-06
## 4    50     88.00    5.288e-04
##
##
## Elapsed time : 1.131618
```

The plot of the residuals or standardized residual of the best model can be used to check if the volatility clusters seen in raw returns have disappeared, indicating the model captured predictable volatility patterns. A well-fitting model should produce a series that looks like random noise.

```
std_residuals <- residuals(arma15.garch11.fit.t, standardize = TRUE)

forecast::autoplot(std_residuals)
```



The histogram of the standardized residuals can be used also to check alignment with the Student's t-distribution assumed in the model. The histogram must be bell-shaped and centered at zero with higher peak and fatter tails than a Normal distribution.



Assuming evrything works well, then we can use the best-fitting model for forecasting volatility.

```
(egarch_forecast <- ugarchforecast(arma15.garch11.fit.t, n.ahead = 10))
```

```
##
## *------------------------------------*
## *       GARCH Model Forecast         *
## *------------------------------------*
## Model: sGARCH
## Horizon: 10
## Roll Steps: 0
## Out of Sample: 0
##
## 0-roll forecast [T0=2013-11-06]:
##          Series   Sigma
## T+1   0.0010144 0.01395
## T+2   0.0012975 0.01400
## T+3  -0.0001491 0.01404
## T+4   0.0009342 0.01409
## T+5   0.0006794 0.01413
```

```
## T+6   0.0006744 0.01418
## T+7   0.0006700 0.01423
## T+8   0.0006661 0.01427
## T+9   0.0006627 0.01432
## T+10  0.0006597 0.01436
```